Guia de Estudo — Prova de API REST

Guia Completo: API REST com Java, Spring Boot, JWT e PostgreSQL

Passo a passo do zero até a API testada: criar o projeto, modelar as entidades, proteger com JWT e documentar com Swagger. Inclui a análise dos erros encontrados na prova anterior.

> **Status da documentação:**
>
> Este guia consolida (1) o passo a passo do curso (VS Code, Maven, Spring Initializr, camadas MVC, Swagger), (2) a documentação de arquitetura com JWT e (3) a prova realizada, no tema **clínica / atendimento infantil** (Responsável, Criança, Psicólogo e Agendamento). Todo o código mostrado aqui já está **corrigido** e alinhado ao MER e ao script SQL da prova.

> **Antes de usar na prova real:**
>
> Eu revisei o código linha a linha, mas **não consegui compilar e executar o projeto neste ambiente** (sem acesso ao Maven Central). Faça um treino completo seguindo este guia uma vez, em casa, para confirmar que tudo roda na sua máquina (Java 25, PostgreSQL 18).

Convenção de cores

- **Verde**: títulos, passos importantes e termos-chave.
- Caixas verdes: dicas e conceitos. Caixas amarelas: atenção. Caixas vermelhas: erro comum que reprova.
- Blocos de código com faixa verde. Linhas com fundo verde-claro são as que **mudaram em relação ao que normalmente se erra**.

Sumário

1. Visão geral e plano de ataque . . . 3
2. Pré-requisitos e ferramentas . . . 5
3. Lendo o enunciado, o MER e o script SQL . . . 7
4. GitHub e clone do repositório . . . 9
5. Criando o projeto Spring Boot . . . 11
6. Dependências extras no pom.xml . . . 14
7. Configurações: application.properties . . . 15
8. Primeira execução . . . 17
9. Estrutura de pacotes . . . 19
10. Camada de modelo (models / entities) . . . 20
11. Camada de repositórios (repositories) . . . 25
12. Camada de serviços (services) . . . 27
13. Camada de controle (controllers) . . . 34
14. Segurança com JWT (login) . . . 39
15. Swagger / OpenAPI . . . 47
16. Executar e testar a API . . . 48
17. Commit e push para o GitHub . . . 50
18. Análise dos erros da prova anterior . . . 51
19. Erros encontrados nos materiais de referência . . . 53
20. Erros comuns e como resolver . . . 54
21. Checklist final antes de entregar . . . 55
22. Adaptando este guia para outra prova . . . 56

1. Visão geral e plano de ataque

Antes de abrir o VS Code, entenda **o que a prova pede** e **em que ordem construir**. A maior causa de perda de tempo é começar a codar sem ter lido o MER e o script SQL.

1.1 Requisitos funcionais da prova

Requisito (enunciado)	O que implementar	Endpoints
Autenticação: login seguro, validar falha, exibir nome do usuário logado, acessar recursos e sair	Spring Security + JWT. Login devolve **token + nome**. Falha devolve **401** com mensagem. Logout descarta o token.	POST /auth/login POST /auth/logout
Responsáveis: cadastrar, listar, buscar, atualizar, excluir, com **busca por termo** na listagem	CRUD completo + parâmetro *termo* que filtra pelo nome.	POST /responsavel/salvar-responsavel GET /responsavel/listar-responsaveis?termo= GET /responsavel/buscar-responsavel/{id} PUT /responsavel/atualizar-responsavel/{id} DELETE /responsavel/deletar-responsavel/{id}
Crianças: cadastrar associada a um responsável, consultar, atualizar, excluir	CRUD; no cadastro, receber **responsavel.id**.	POST /crianca/salvar-crianca GET /crianca/listar-criancas GET /crianca/buscar-crianca/{id} PUT /crianca/atualizar-crianca/{id} DELETE /crianca/deletar-crianca/{id}
Agendamentos: listar **ordenados por data** com dados da criança, do responsável e do psicólogo	Método do repository com **OrderBy**; o JSON já traz criança, responsável e psicólogo.	GET /agendamento/listar-agendamentos POST /agendamento/salvar-agendamento GET /agendamento/buscar-agendamento/{id} PUT /agendamento/atualizar-agendamento/{id} DELETE /agendamento/deletar-agendamento/{id}

Infraestrutura exigida: SGBD **PostgreSQL 18** e linguagem **Java 25**.

1.2 Arquitetura em camadas (MVC expandido)

O curso usa o MVC com duas camadas extras, muito comum em Spring Boot: **Controller** recebe a requisição HTTP, **Service** aplica as regras de negócio, **Repository** conversa com o banco via JPA e **Model** representa a tabela.

Camada	Pacote	Nome da classe	Responsabilidade
Model	models	Responsavel	Entidade JPA = tabela do banco
Repository	repositories	ResponsavelRepository	Interface que estende JpaRepository (CRUD pronto)
Service	services	ResponsavelService	Lógica de cada operação e criptografia de senha
Controller	controllers	ResponsavelController	Endpoints REST (GET, POST, PUT, DELETE)
DTO	dtos	LoginResponseDTO	Objeto de entrada/saída do login
Config	configs	SecurityConfig, JwtService	Segurança, JWT, CORS e Swagger

Fluxo de uma requisição

```
Cliente (Swagger / Postman / Front-end)
        |  HTTP + JSON  (+ Authorization: Bearer <token>)
        v
JwtAuthFilter  ->  valida o token e preenche o SecurityContext
        v
Controller     ->  recebe @RequestBody / @PathVariable / @RequestParam
        v
Service        ->  regras de negócio (buscar, validar, criptografar)
        v
Repository     ->  JpaRepository (SELECT / INSERT / UPDATE / DELETE)
        v
PostgreSQL     ->  tabelas do script da prova
```

1.3 Ordem de trabalho recomendada

1. Ler enunciado, **MER** e **script SQL** (seção 3). Anotar nomes exatos de tabelas e colunas.
2. Criar repositório no GitHub, clonar no VS Code, instalar extensões (seções 2.1 e 4).
3. Gerar o projeto Spring Boot e adicionar JJWT e Springdoc ao pom.xml (seções 5 e 6).
4. Configurar o application.properties e rodar uma primeira vez (seções 7 e 8).
5. Criar os pacotes e as **entidades** (models), conferindo cada @Table e @Column com o script.
6. Criar repositories, services e controllers, **um CRUD por vez**.
7. Implementar a segurança JWT e o login (seção 14).
8. Testar no Swagger/Postman (seção 16).
9. Fazer commit e push (seção 17) e passar pelo checklist final (seção 21).

> **Regra de ouro**
>
> Se o projeto não sobe ou o endpoint não responde, o problema quase sempre está em um destes 3 pontos: **(1)** nome de tabela/coluna diferente do script, **(2)** configuração do banco no application.properties, **(3)** senha gravada sem criptografia. Veja a seção 20.

2. Pré-requisitos e ferramentas

Confira tudo isso **antes** do dia da prova.

Ferramenta	Versão / observação	Para quê
JDK (Java)	25 (confira com `java --version` no terminal)	Compilar e executar a API
Maven	Já vem no projeto (`mvnw`); precisa de JAVA_HOME configurado	Gerenciar dependências e build
Visual Studio Code	Última versão	Editor de código
PostgreSQL	18, usuário `postgres`, senha combinada (ex.: aluno)	Banco de dados
Git + conta GitHub	Git Bash instalado	Versionamento e entrega
Postman (opcional)	Pode usar só o Swagger UI	Testar requisições com token

2.1 Extensões do VS Code

Abra o painel de Extensões (ícone de quadrados) e instale as do curso:

- **Java**: Extension Pack for Java, Project Manager for Java, Maven for Java, Debugger for Java, Language Support for Java.
- **Spring**: Spring Boot Extension Pack e Spring Initializr Java Support.
- **Git**: GitHub Repositories, Remote Repositories, Git History, Git Graph e Start git-bash.
- **Formatação**: Prettier — Code formatter e Markdown Preview Enhanced (opcionais).

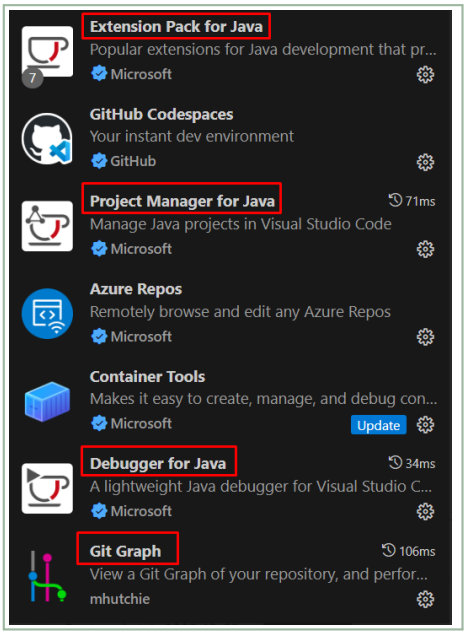


Figura 1 — Extensões de Java

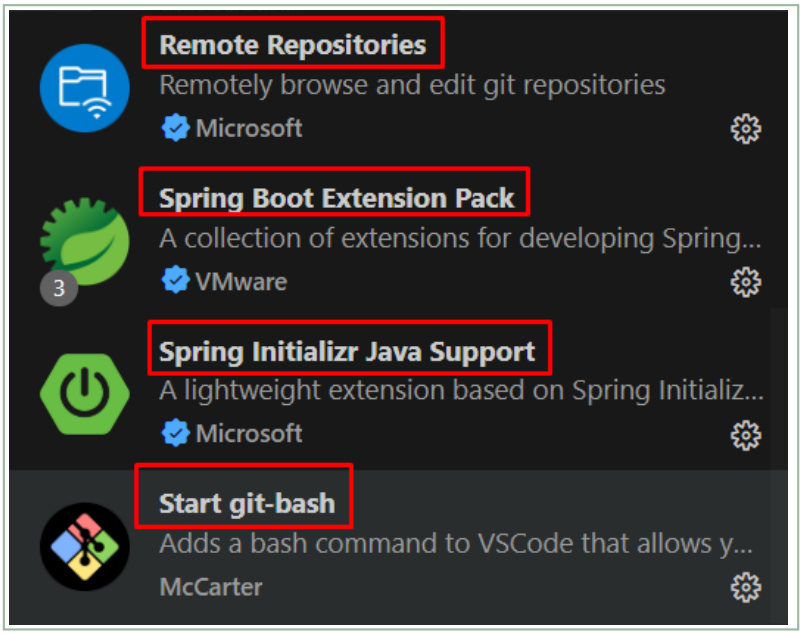


Figura 2 — Extensões de Spring e Git

> **Dica**
>
> Sempre que o VS Code mostrar uma caixa no canto inferior direito pedindo para analisar o projeto (ex.: *null annotation types*), clique em **Enable**.

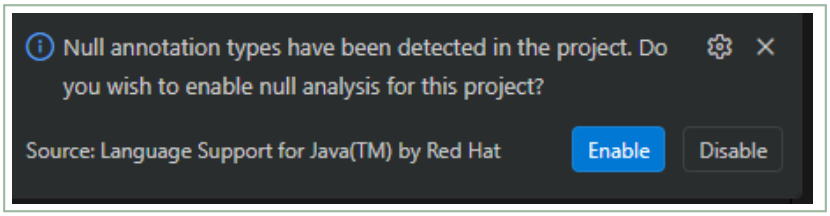


Figura 3 — Caixa que aparece no canto inferior direito: clique em Enable

3. Lendo o enunciado, o MER e o script SQL

Este é o passo mais barato e o que mais evita erro: **tudo que você escrever em Java deve espelhar o banco**.

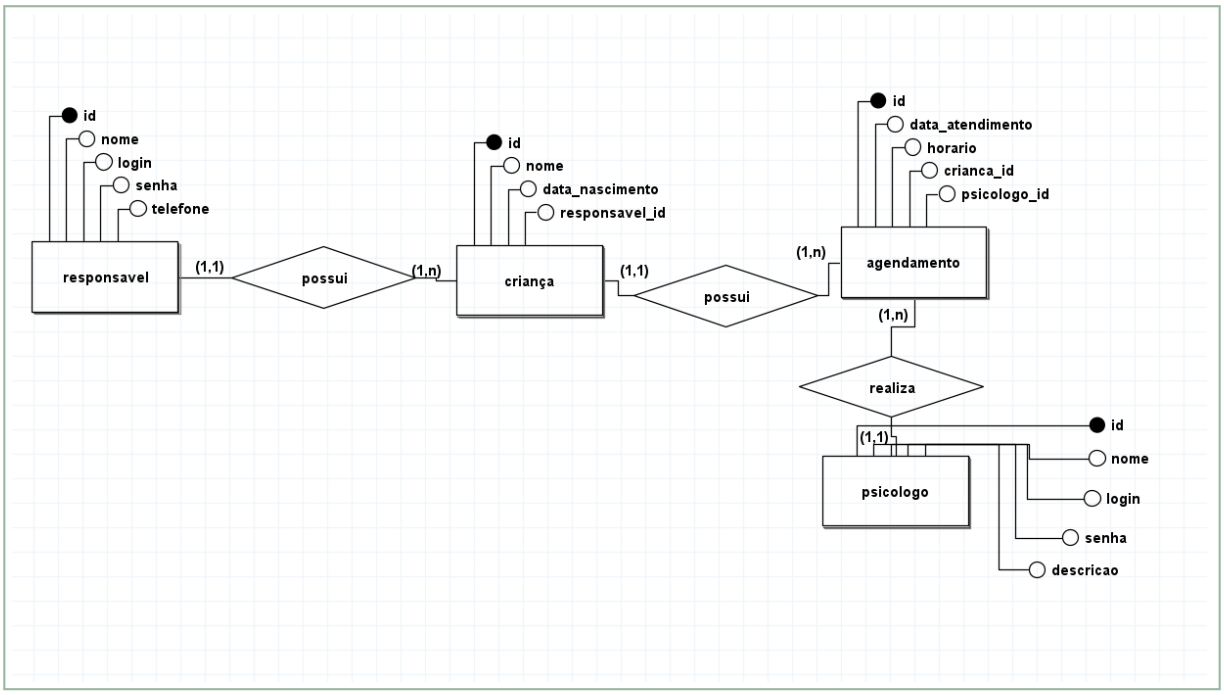


Figura 4 — MER da prova: Responsável, Criança, Agendamento e Psicólogo

3.1 Entidades, atributos e relacionamentos

Entidade	Atributos (colunas)	Relacionamento
responsavel	id (PK), nome, login, senha, telefone	Um responsável possui **várias** crianças (1,n)
crianca	id (PK), nome, data_nascimento, responsavel_id (FK)	Cada criança tem **um** responsável e pode ter **vários** agendamentos
agendamento	id (PK), data_atendimento, horario, crianca_id (FK), psicologo_id (FK)	Pertence a **uma** criança e a **um** psicólogo
psicologo	id (PK), nome, login, senha, descricao	Realiza **vários** agendamentos; também é o usuário que faz login

> **Como traduzir o MER em JPA**
>
> O lado **"n"** (quem tem a chave estrangeira) recebe `@ManyToOne` + `@JoinColumn`. Criança tem `@ManyToOne Responsavel`; Agendamento tem `@ManyToOne Crianca` e `@ManyToOne Psicologo`. Nunca use **`@OneToOne`** aqui: ele cria uma restrição de unicidade e impede que a mesma criança tenha dois agendamentos.

3.2 Regras de mapeamento: script SQL → classe Java

Item	Regra	Exemplo
Nome da tabela	**@Table(name)** deve ser **idêntico** ao CREATE TABLE do script (inclusive plural)	@Table(name = "criancas")
Nome da coluna	**@Column(name)** idêntico ao script	@Column(name = "data_atendimento")
Atributo Java	camelCase, sem underline	private LocalDate dataAtendimento;
Tipo	Deve casar com o tipo da coluna	DATE → LocalDate, TIME → LocalTime, VARCHAR → String, SERIAL/INT → Integer
Chave primária	SERIAL → `@GeneratedValue(strategy = IDENTITY)`	@Id + @GeneratedValue
Chave estrangeira	Objeto da entidade relacionada, não o id solto	private Responsavel responsavel;

> **Por que isso importa**
>
> Com `spring.jpa.hibernate.ddl-auto=update`, se o nome da tabela na entidade for diferente do script, o Hibernate **cria uma tabela nova e vazia** com o nome da entidade e ignora a sua tabela com dados. O resultado é uma API que "funciona" mas só lista vazio. Foi exatamente o que aconteceu na prova anterior (seção 18).

4. GitHub e clone do repositório

O repositório é a forma de entrega. Se já tem uma organização de estudos no GitHub, use-a.

5.1 Criar o repositório

1. No GitHub, clique na sua foto de perfil → **Your organizations** e escolha a organização do curso.
2. Clique em **New repository**. Informe o nome, mantenha **Private** e marque **Add a README file**.
3. Clique em **Create repository**.

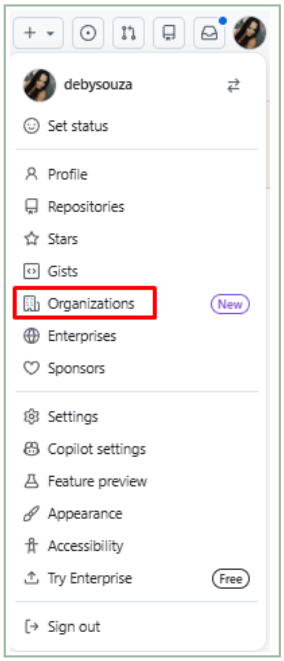


Figura 5 — Menu do perfil → Organizations

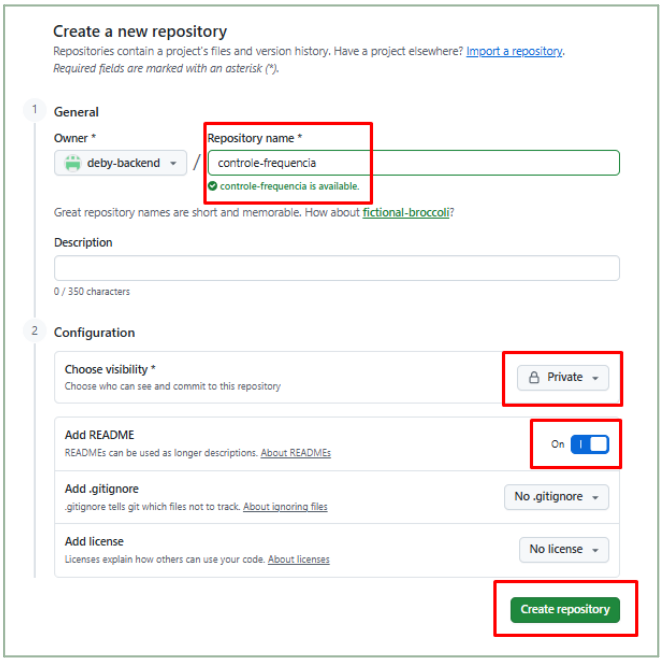


Figura 6 — Tela de novo repositório

5.2 Clonar no VS Code

1. No repositório, clique em **Code → HTTPS** e copie o link com o botão de copiar.
2. Abra o VS Code vazio, vá em **Explorer** e clique em **Clone Repository**.
3. Cole o link e pressione **ENTER**. Escolha a pasta onde o projeto ficará (use um nome **sem espaços ou acentos**, ex.: `lucas`).
4. Aguarde o clone e clique em **Open** na caixa que aparecer.

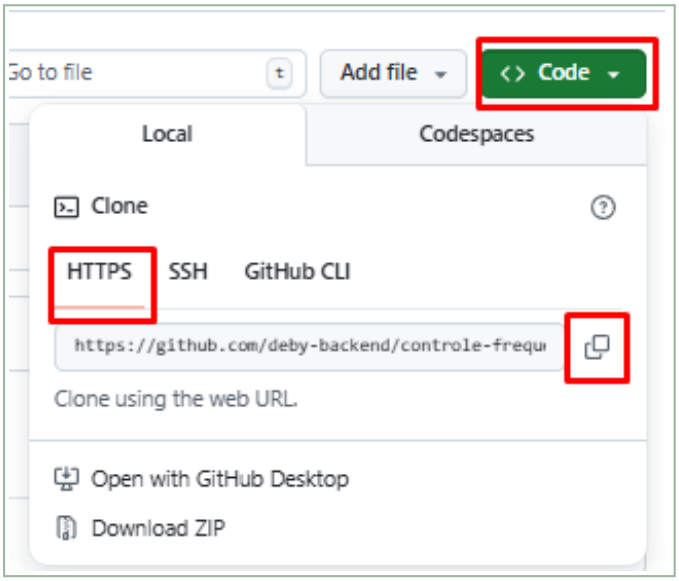


Figura 7 — Code → HTTPS → copiar

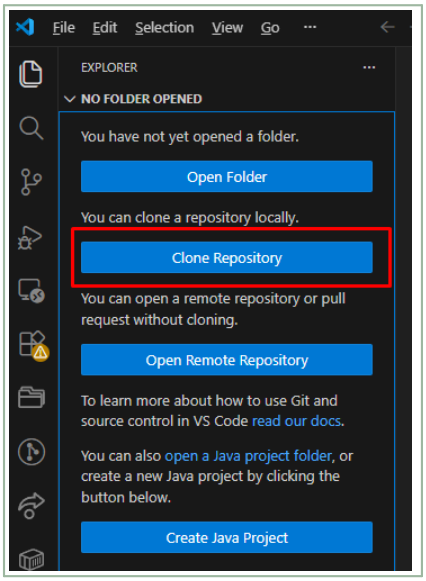


Figura 8 — Clone Repository

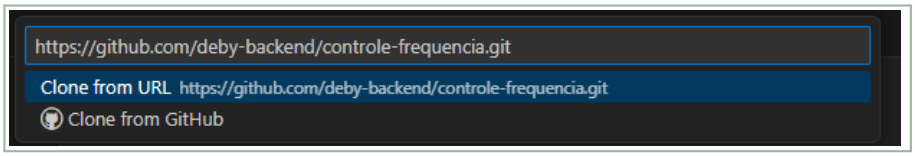


Figura 9 — Colar a URL e pressionar ENTER

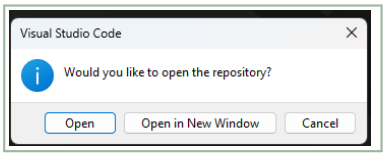


Figura 10 — Open para abrir o repositório

> **Atenção**
>
> Evite pastas dentro do OneDrive com nomes longos, acentos ou espaços: o Maven e o Java podem falhar com caminhos assim. Prefira algo como `C:\dev\seunome`.

5. Criando o projeto Spring Boot

Com o repositório aberto no VS Code, clique com o botão direito em um espaço vazio abaixo do README e escolha **New Java Project**.

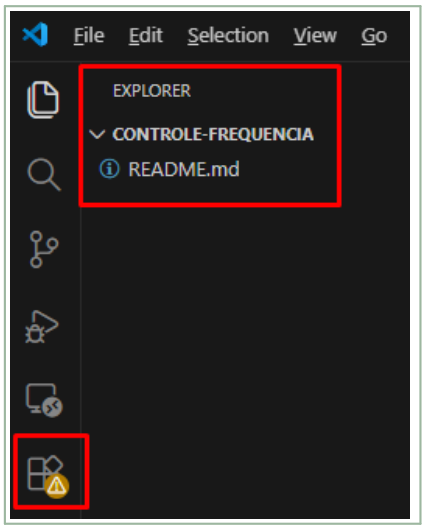


Figura 11 — Explorer com o README

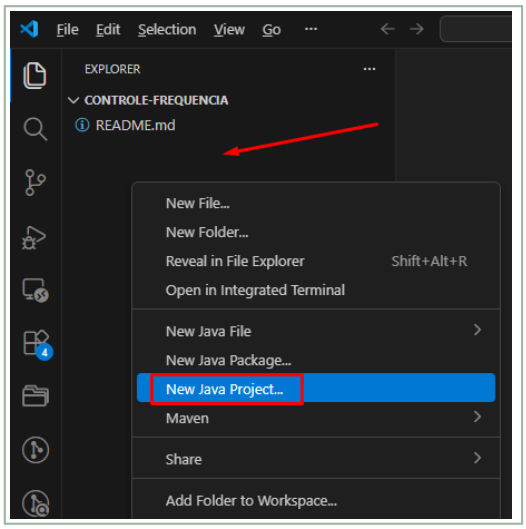


Figura 12 — Menu: New Java Project...

O assistente do Spring Initializr faz uma pergunta por vez. Responda assim:

Pergunta	Resposta	Por quê
Project type	**Spring Boot**	Framework da disciplina
Build tool	**Maven Project**	Recomendado para APIs
Spring Boot version	Versão **mais atual sem sufixo** (sem M, RC ou SNAPSHOT)	Versões com sufixo são instáveis
Language	**Java**	
Group Id	URL ao contrário, ex.: `com.treino`	Boa prática do ecossistema Java
Artifact Id	nome do projeto sem caracteres especiais, ex.: `senai`	Vira o nome do pacote base
Package name	conferir: `com.treino.senai`	Digitar errado quebra o scan de componentes
Packaging	**Jar**	Executável
Java version	**25**	Mesma do java --version

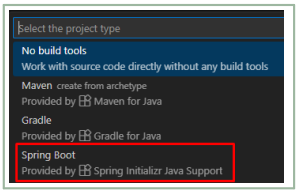


Figura 13 — Spring Boot

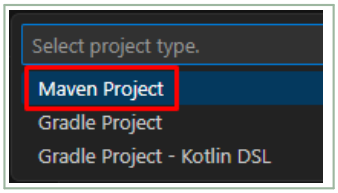


Figura 14 — Maven Project

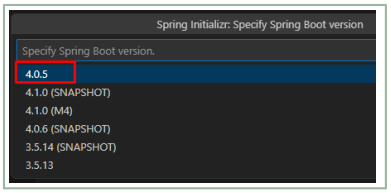


Figura 15 — Versão mais atual

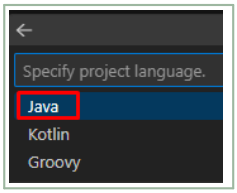


Figura 16 — Java

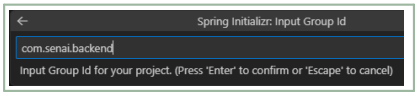


Figura 17 — Group Id

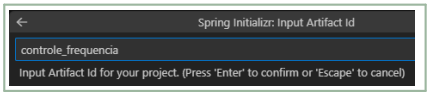


Figura 18 — Artifact Id

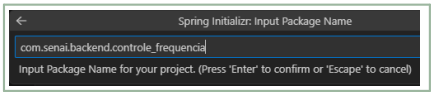


Figura 19 — Package Name

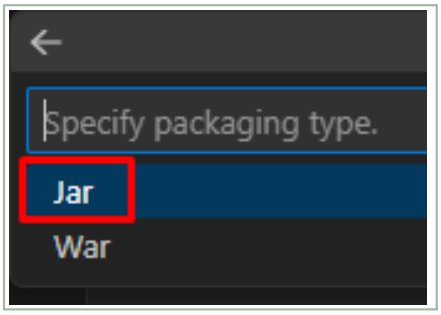


Figura 20 — Packaging Jar

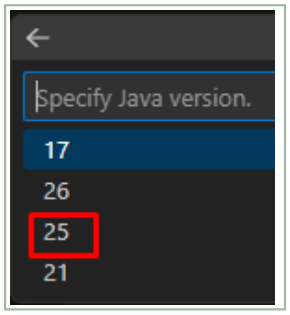


Figura 21 — Versão do Java (25)

6.1 Dependências

Na lista de dependências marque as abaixo (digite o nome para filtrar e use a barra de espaço para marcar), depois ENTER:

- **Spring Web** — controllers REST.
- **Spring Data JPA** — persistência com Hibernate.
- **PostgreSQL Driver** — driver JDBC.
- **Spring Security** — autenticação e autorização.
- **Spring Boot DevTools** — reinício automático.
- Opcional: **Validation**. Não é necessário marcar Lombok, Thymeleaf nem Actuator (a prova anterior tinha esses extras sem uso).

Confirme a pasta onde salvar: deve ser **a mesma que contém a pasta oculta .git** (o repositório clonado). Por fim clique em **Add to Workspace**.

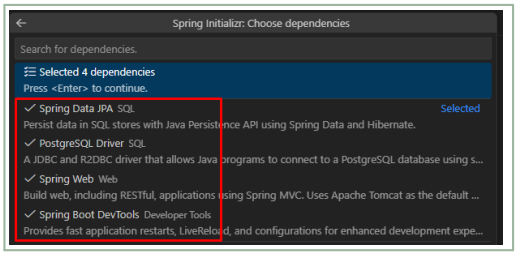


Figura 22 — Seleção de dependências

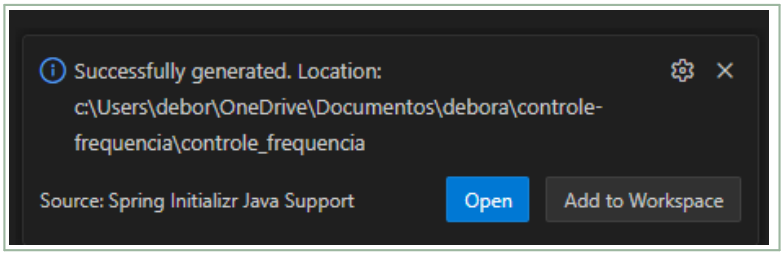


Figura 23 — Projeto gerado: Add to Workspace

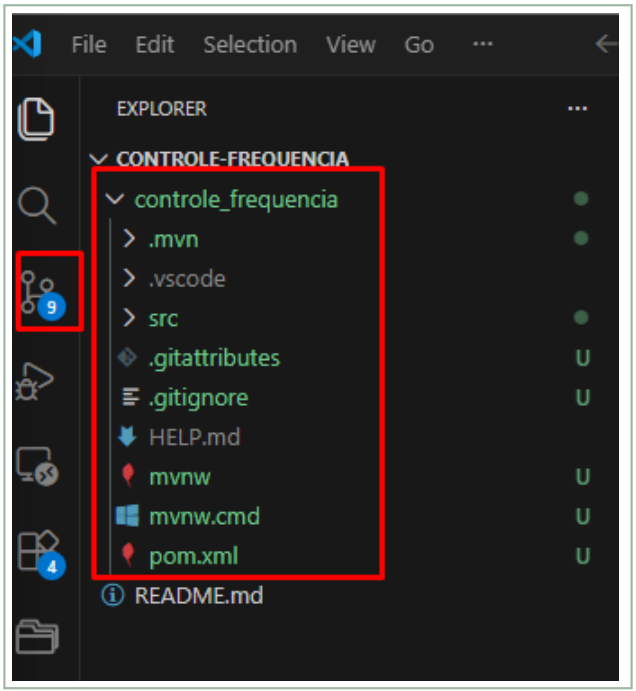


Figura 24 — Árvore do projeto criado e alterações do Git desde o clone

6. Dependências extras no pom.xml

O Initializr já adicionou Web, JPA, PostgreSQL, Security e DevTools. Faltam duas: **JJWT** (gerar e validar o token) e **Springdoc** (Swagger UI). Abra o `pom.xml` e cole as dependências abaixo **dentro da tag** `<dependencies>`, antes de `</dependencies>`.

pom.xml — dependências a adicionar

```
<!-- JWT (JSON Web Token) -->
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.6</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>

<!-- Swagger / OpenAPI (versão 3.x para Spring Boot 4.x) -->
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>3.0.3</version>
</dependency>
```

Salve (Ctrl + S). Se o VS Code perguntar *"A build file was modified. Do you want to synchronize the Java classpath?"*, clique em **Yes / Always**. Se não aparecer, use **Ctrl + Shift + P → Java: Clean Java Language Server Workspace**.

Biblioteca	Função	Observação
jjwt-api	Interfaces para criar e ler tokens (Jwts.builder, Jwts.parser)	Usada no código
jjwt-impl	Implementação dos algoritmos HMAC/RSA	scope runtime
jjwt-jackson	Converte o conteúdo do token em JSON	scope runtime
springdoc-openapi-starter-webmvc-ui	Gera o Swagger UI a partir dos controllers	A versão **depende da versão do Spring Boot**

> **Erro comum: versão do Springdoc incompatível**
>
> Springdoc **2.x** serve para Spring Boot **3.x**; Spring Boot **4.x** exige Springdoc **3.x**. A prova anterior usava Boot 4.1.1 com Springdoc 2.2.0, combinação que não é suportada e costuma quebrar a inicialização ou o Swagger UI. Confira a versão do Boot no topo do pom (`<parent>`) e escolha a linha correspondente. Se aparecer erro com o Boot 4.1, procure no GitHub do springdoc-openapi a release mais recente da série 3.x.

Também não é preciso manter, como na prova anterior, as dependências *spring-boot-starter* duplicada, *jakarta.persistence-api*, Thymeleaf, Actuator e Lombok sem uso: quanto mais dependências, maior a chance de conflito.

7. Configurações: application.properties

Arquivo em `src/main/resources/application.properties`. Substitua todo o conteúdo por:

application.properties

```
spring.application.name=senai
server.port=8080

# Banco de dados (o banco "clinica" precisa existir)
spring.datasource.url=jdbc:postgresql://localhost:5432/clinica
spring.datasource.username=postgres
spring.datasource.password=aluno
spring.datasource.driver-class-name=org.postgresql.Driver

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

# JWT: segredo com pelo menos 32 caracteres; validade em milissegundos (24 h)
jwt.secret=ChaveSecretaDaClinicaPsicologia2026ComMaisDe32Caracteres
jwt.expiration=86400000
```

Propriedade	O que faz	Cuidado
server.port	Porta do Tomcat embutido	Deve ser a mesma usada no Swagger/Postman (8080 vs 8081)
spring.datasource.url	Endereço do banco: jdbc:postgresql://host:porta/**nome_do_banco**	O nome do banco precisa existir
spring.datasource.username / password	Credenciais do PostgreSQL	Senha definida na instalação
spring.jpa.hibernate.ddl-auto=update	Cria/ajusta tabelas a partir das entidades	Se o nome da tabela divergir do script, cria tabela nova vazia
spring.jpa.show-sql=true	Mostra o SQL gerado no console	Ótimo para depurar
jwt.secret	Chave de assinatura do token	**Mínimo 32 caracteres**, senão ocorre WeakKeyException
jwt.expiration	Tempo de vida do token em ms	86400000 = 24 h; 1200000 = 20 min

Dica de depuração

Troque temporariamente `ddl-auto=update` por `ddl-auto=validate`. Com *validate* o Hibernate não cria nada: se alguma tabela ou coluna da entidade não existir no banco, a aplicação falha na hora de subir e diz exatamente qual. É o jeito mais rápido de achar um @Table ou @Column errado. Depois volte para *update*, como pede o material do curso.

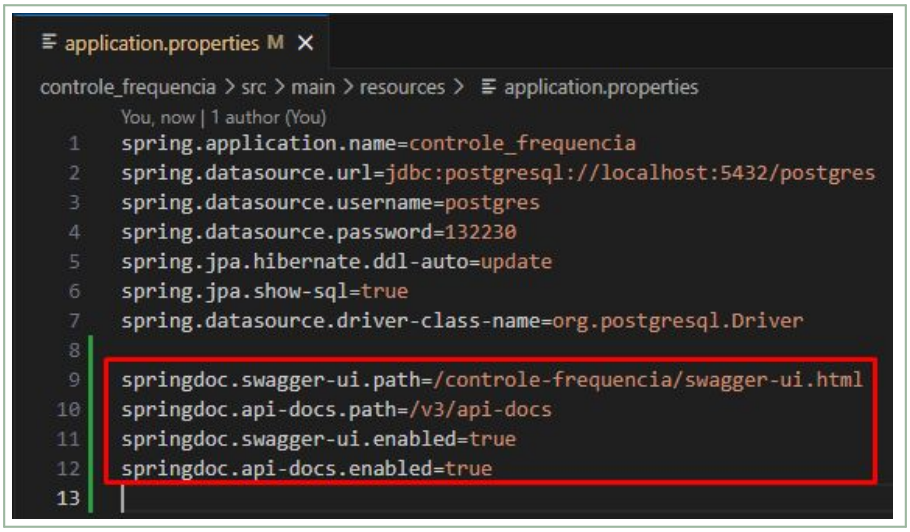


Figura 25 — application.properties no VS Code (exemplo do material de apoio)

> **Segurança**
>
> Em projeto real, senha do banco e segredo do JWT ficam em variáveis de ambiente, nunca no repositório. Na prova, deixe como está e ajuste a senha do postgres para a da máquina.

8. Primeira execução

Antes de escrever qualquer código, rode o projeto vazio para provar que Java, Maven e VS Code estão funcionando.

1. Abra `src/main/java/com/treino/senai/SenaiApplication.java`.
2. Dentro do `main`, depois do `SpringApplication.run(...)`, adicione um `System.out.println` (equivale ao *escreva* do Portugol).
3. Clique em **Run Java** (canto superior direito) ou no **Run** acima do método main.

SenaiApplication.java

```java
@SpringBootApplication
public class SenaiApplication {

    public static void main(String[] args) {
        SpringApplication.run(SenaiApplication.class, args);
        System.out.println("Aplicação iniciada com sucesso!");
    }
}
```

> **Se a aplicação não subir**
>
> Leia a **última linha do erro** no terminal: *password authentication failed* = senha do postgres errada; *database does not exist* = o banco do script ainda não existe; *Port 8080 was already in use* = outra instância rodando (pare-a com o quadrado vermelho do terminal). Mais casos na seção 20.

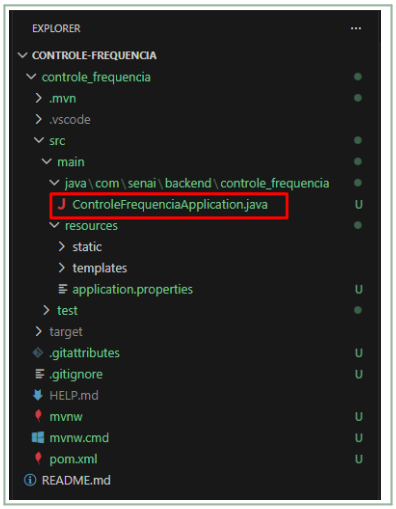


Figura 26 — Arquivo da aplicação

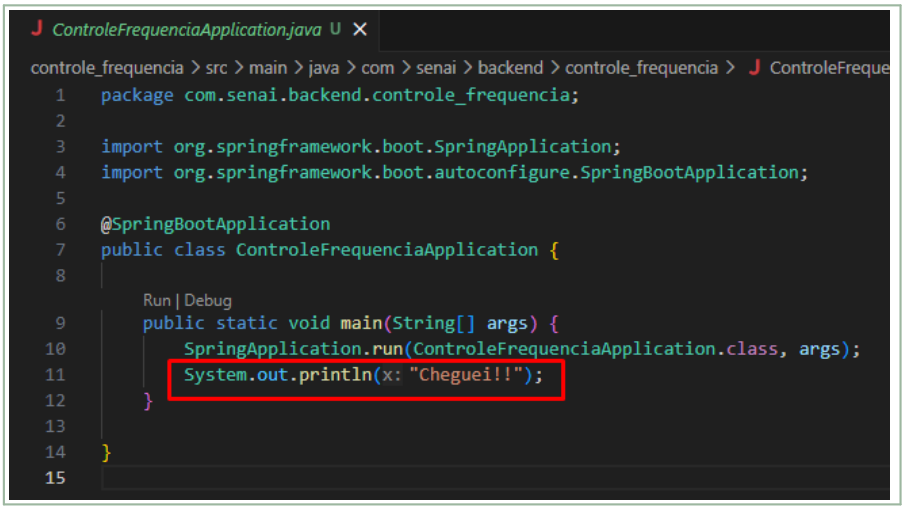


Figura 27 — println e botão Run Java

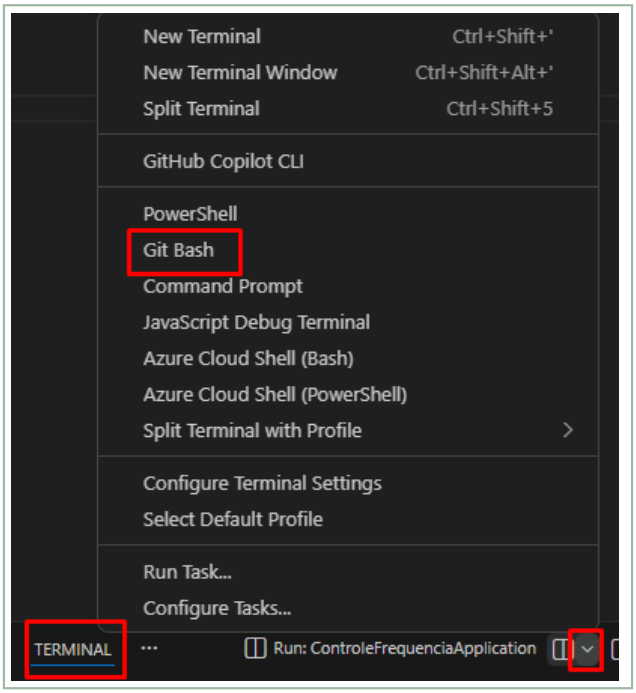


Figura 28 — Menu de execução do VS Code

9. Estrutura de pacotes

Pacotes são pastas que representam as camadas. **Boa prática:** minúsculo e no plural. Clique com o botão direito em `com.treino.senai` → **New Folder** e crie cada um dentro dele.

Árvore final do projeto

```
src/main/java/com/treino/senai/
|-- SenaiApplication.java
|-- configs/        SecurityConfig, JwtAuthFilter, CustomUserDetailsService, Swagger
|-- controllers/    AuthController, ResponsavelController, CriancaController,
|                   PsicologoController, AgendamentoController
|-- dtos/           LoginRequestDTO, LoginResponseDTO
|-- models/         Responsavel, Crianca, Psicologo, Agendamento
|-- repositories/   ResponsavelRepository, CriancaRepository,
|                   PsicologoRepository, AgendamentoRepository
|-- services/       ResponsavelService, CriancaService, PsicologoService,
|                   AgendamentoService, JwtService
src/main/resources/
|-- application.properties
```

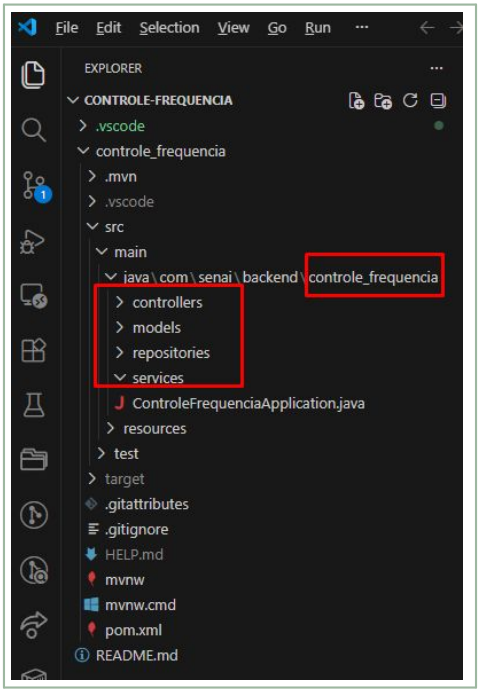


Figura 29 — Pacotes dentro do pacote base (exemplo do material de apoio)

Padronização que evita erro

Classes: PascalCase, no singular e sem acento (Responsavel, não Responsável). **Pacotes**: minúsculos (configs, não Security). O projeto da prova anterior tinha as pastas `Security` e enuns; prefira `configs` (e `enums`, se precisar). Todo pacote deve estar **dentro** de com.treino.senai, senão o Spring não encontra os componentes.

10. Camada de modelo (models / entities)

Cada entidade é uma classe anotada com `@Entity` que espelha uma tabela. Siga a tabela de regras da seção 3.2: nomes e tipos **idênticos ao script SQL**.

Anotação / termo	Função
package	Localização do arquivo dentro do projeto
import	Carrega as bibliotecas usadas na classe
@Entity	Define a classe como entidade JPA
@Table(name = "...")	Nome da tabela no banco (igual ao script)
@Id	Define a chave primária (PK)
@GeneratedValue(strategy = IDENTITY)	Valor gerado pelo banco (SERIAL)
@Column(name = "...")	Nome da coluna no banco
@ManyToOne + @JoinColumn(name = "x_id")	Chave estrangeira: muitos registros desta entidade para um da outra
@JsonProperty(access = WRITE_ONLY)	Aceita o campo na entrada do JSON, mas **nunca o devolve** (usado em senha)
private / public	Modificadores de acesso: atributos privados, métodos públicos (encapsulamento)
Construtor default e parametrizado	Default é exigido pelo JPA; parametrizado facilita criar objetos
Getters e Setters	Acesso controlado aos atributos privados

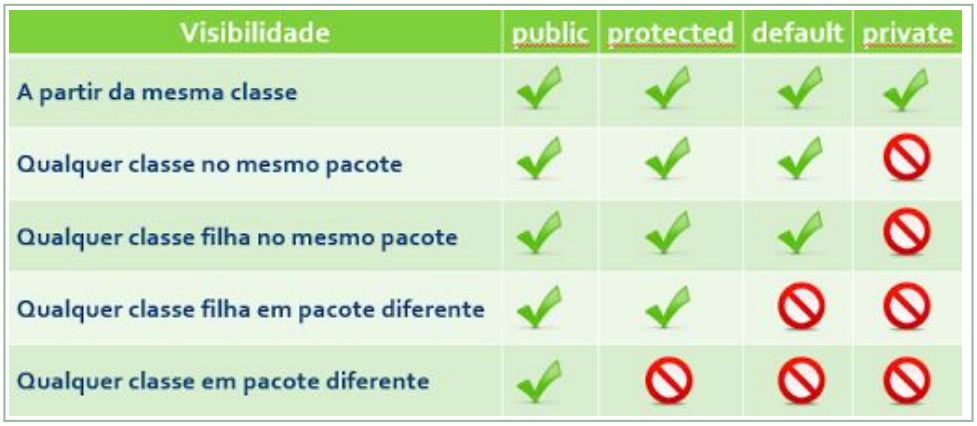


Figura 30 — Modificadores de acesso (material de apoio)

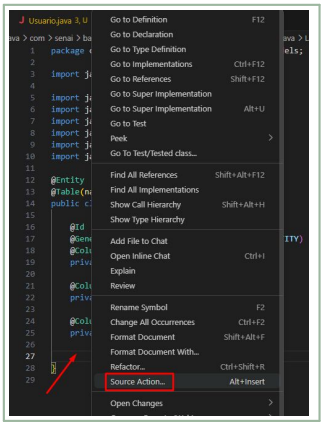


Figura 31 — Botão direito → Source Action...

Para gerar construtores e getters/setters sem digitar: clique com o botão direito na linha vazia antes da chave final da classe → **Source Action → Generate Constructors** (uma vez sem selecionar nenhum campo para o construtor default, outra com todos) e depois **Generate Getters and Setters** marcando todos os campos.

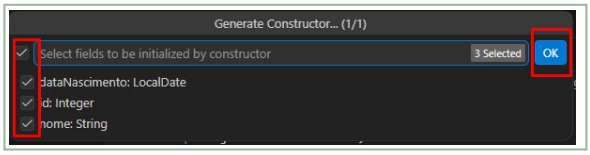


Figura 32 — Generate Constructors

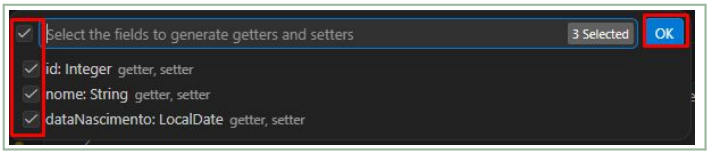


Figura 33 — Generate Getters and Setters

10.1 Responsavel.java

models/Responsavel.java

```java
package com.treino.senai.models;

import com.fasterxml.jackson.annotation.JsonProperty;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "responsaveis")
public class Responsavel {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Integer id;

    @Column(name = "nome")
    private String nome;

    @Column(name = "login")
    private String login;

    // entra no JSON de cadastro, mas não é devolvido nas respostas
    @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
    @Column(name = "senha")
    private String senha;

    @Column(name = "telefone")
    private String telefone;

    public Responsavel() {
    }

    public Responsavel(Integer id, String nome, String login, String senha, String telefone) {
        this.id = id;
        this.nome = nome;
        this.login = login;
        this.senha = senha;
        this.telefone = telefone;
    }

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public String getLogin() { return login; }
    public void setLogin(String login) { this.login = login; }
    public String getSenha() { return senha; }
    public void setSenha(String senha) { this.senha = senha; }
    public String getTelefone() { return telefone; }
    public void setTelefone(String telefone) { this.telefone = telefone; }
}
```

10.2 Crianca.java

models/Crianca.java

```java
package com.treino.senai.models;

import java.time.LocalDate;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;

@Entity
@Table(name = "criancas")
public class Crianca {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Integer id;

    @Column(name = "nome")
    private String nome;

    @Column(name = "data_nascimento")
    private LocalDate dataNascimento;

    @ManyToOne
    @JoinColumn(name = "responsavel_id")
    private Responsavel responsavel;

    public Crianca() {
    }

    public Crianca(Integer id, String nome, LocalDate dataNascimento, Responsavel responsavel) {
        this.id = id;
        this.nome = nome;
        this.dataNascimento = dataNascimento;
        this.responsavel = responsavel;
    }

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public LocalDate getDataNascimento() { return dataNascimento; }
    public void setDataNascimento(LocalDate dataNascimento) {
        this.dataNascimento = dataNascimento;
    }
    public Responsavel getResponsavel() { return responsavel; }
    public void setResponsavel(Responsavel responsavel) { this.responsavel = responsavel; }
}
```

10.3 Psicologo.java

O psicólogo é quem faz login. Aqui a entidade é **simples** (sem implementar UserDetails): a conversão para o formato do Spring Security é feita no `CustomUserDetailsService` (seção 14). Isso evita que o JSON exponha campos como authorities, enabled e password.

models/Psicologo.java

```java
package com.treino.senai.models;

import com.fasterxml.jackson.annotation.JsonProperty;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "psicologo")
public class Psicologo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Integer id;

    @Column(name = "nome")
    private String nome;

    @Column(name = "login", unique = true)
    private String login;

    @JsonProperty(access = JsonProperty.Access.WRITE_ONLY)
    @Column(name = "senha")
    private String senha;

    // perfil de acesso: ADMIN ou OPERADOR
    @Column(name = "descricao")
    private String descricao;

    public Psicologo() {
    }

    public Psicologo(Integer id, String nome, String login, String senha, String descricao) {
        this.id = id;
        this.nome = nome;
        this.login = login;
        this.senha = senha;
        this.descricao = descricao;
    }

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public String getLogin() { return login; }
    public void setLogin(String login) { this.login = login; }
    public String getSenha() { return senha; }
    public void setSenha(String senha) { this.senha = senha; }
    public String getDescricao() { return descricao; }
    public void setDescricao(String descricao) { this.descricao = descricao; }
}
```

10.4 Agendamento.java

models/Agendamento.java

```java
package com.treino.senai.models;

import java.time.LocalDate;
import java.time.LocalTime;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;

@Entity
@Table(name = "agendamentos")
public class Agendamento {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private Integer id;

    @Column(name = "data_atendimento")
    private LocalDate dataAtendimento;

    @Column(name = "horario")
    private LocalTime horario;

    @ManyToOne
    @JoinColumn(name = "crianca_id")
    private Crianca crianca;

    @ManyToOne
    @JoinColumn(name = "psicologo_id")
    private Psicologo psicologo;

    public Agendamento() {
    }

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }
    public LocalDate getDataAtendimento() { return dataAtendimento; }
    public void setDataAtendimento(LocalDate dataAtendimento) {
        this.dataAtendimento = dataAtendimento;
    }
    public LocalTime getHorario() { return horario; }
    public void setHorario(LocalTime horario) { this.horario = horario; }
    public Crianca getCrianca() { return crianca; }
    public void setCrianca(Crianca crianca) { this.crianca = crianca; }
    public Psicologo getPsicologo() { return psicologo; }
    public void setPsicologo(Psicologo psicologo) { this.psicologo = psicologo; }
}
```

O que foi corrigido em relação à prova

1) @Table com o nome do script (*agendamentos*, não *agendamento*). **2)** Coluna *horario* (a prova mapeava *hora_atendimento*, que não existe). **3)** @ManyToOne no lugar de @OneToOne. **4)** LocalDate/LocalTime no lugar de java.sql.Date/Time e de String para datas. **5)** Atributos em camelCase (dataAtendimento).

11. Camada de repositórios (repositories)

Repositórios são **interfaces**: funcionam como contratos. Ao estender `JpaRepository<Entidade, TipoDoId>` a interface já herda *save*, *findById*, *findAll*, *deleteById*, *existsById*, *count*. Nome: **Entidade + Repository**. Só criamos método novo quando o enunciado pede algo além do CRUD.

repositories/ResponsavelRepository.java

```java
package com.treino.senai.repositories;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.treino.senai.models.Responsavel;

@Repository
public interface ResponsavelRepository extends JpaRepository<Responsavel, Integer> {

    // busca por termo: o Spring monta o SELECT ... LIKE pelo nome do método
    List<Responsavel> findByNomeContainingIgnoreCase(String termo);
}
```

repositories/CriancaRepository.java

```java
package com.treino.senai.repositories;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.treino.senai.models.Crianca;

@Repository
public interface CriancaRepository extends JpaRepository<Crianca, Integer> {
}
```

repositories/PsicologoRepository.java

```java
package com.treino.senai.repositories;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.treino.senai.models.Psicologo;

@Repository
public interface PsicologoRepository extends JpaRepository<Psicologo, Integer> {

    // usado no login
    Optional<Psicologo> findByLogin(String login);
}
```

repositories/AgendamentoRepository.java

```java
package com.treino.senai.repositories;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.treino.senai.models.Agendamento;

@Repository
public interface AgendamentoRepository extends JpaRepository<Agendamento, Integer> {

    // lista ordenada por data (e horário). Já traz criança, responsável e psicólogo
    List<Agendamento> findAllByOrderByDataAtendimentoAscHorarioAsc();
}
```

Como ler o nome do método

findAllByOrderByDataAtendimentoAscHorarioAsc = buscar todos, ordenar por *dataAtendimento* crescente e depois por *horario* crescente. **findByNomeContainingIgnoreCase** = buscar onde o *nome* contém o texto, ignorando maiúsculas. Os nomes usam os **atributos Java** (dataAtendimento), não as colunas do banco. Como o agendamento tem @ManyToOne para criança e psicólogo, o JSON já mostra os dados deles, e a criança mostra o responsável.

12. Camada de serviços (services)

Os serviços definem a lógica de cada operação. Todos seguem o **mesmo molde** do curso (UsuarioService/PautaService): *contar*, *buscar*, *listar*, *deletar*, *cadastrar*, *atualizar*. Anotação `@Service`; `@Autowired` traz o repositório para dentro da classe. Nome: **Entidade + Service**.

Método	Retorno	O que faz
contarX()	Long	repository.count()
buscarX(id)	Entidade	repository.findById(id).get()
listarXs()	List	repository.findAll()
deletarX(id)	Boolean	Se existsById(id): deleteById e true; senão false
cadastrarX(obj)	Entidade	repository.save(obj)
atualizarX(id, obj)	Entidade	Busca o existente, troca só o que veio diferente de null e salva

Atenção no atualizar

Busque o registro pelo **id da URL** e altere os campos dele. Não use `setId(obj.getId())` (erro da prova anterior): se o JSON não tiver id, o save() cria um registro novo.

services/ResponsavelService.java

```java
package com.treino.senai.services;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import com.treino.senai.models.Responsavel;
import com.treino.senai.repositories.ResponsavelRepository;

@Service
public class ResponsavelService {

    @Autowired
    private ResponsavelRepository responsavelRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    public Long contarResponsaveis() {
        return responsavelRepository.count();
    }

    public Responsavel buscarResponsavel(Integer id) {
        return responsavelRepository.findById(id).get();
    }

    // sem termo lista todos; com termo filtra pelo nome
    public List<Responsavel> listarResponsaveis(String termo) {
        if (termo == null || termo.isBlank()) {
            return responsavelRepository.findAll();
        }
        return responsavelRepository.findByNomeContainingIgnoreCase(termo);
    }

    public Boolean deletarResponsavel(Integer id) {
        if (responsavelRepository.existsById(id)) {
            responsavelRepository.deleteById(id);
            return true;
        }
        return false;
    }

    public Responsavel cadastrarResponsavel(Responsavel responsavel) {
        responsavel.setSenha(passwordEncoder.encode(responsavel.getSenha()));
        return responsavelRepository.save(responsavel);
    }

    public Responsavel atualizarResponsavel(Integer id, Responsavel responsavel) {
        Responsavel responsavelRecuperado = buscarResponsavel(id);
        if (responsavelRecuperado != null) {
            if (responsavel.getNome() != null) {
                responsavelRecuperado.setNome(responsavel.getNome());
            }
            if (responsavel.getLogin() != null) {
                responsavelRecuperado.setLogin(responsavel.getLogin());
            }
            if (responsavel.getTelefone() != null) {
                responsavelRecuperado.setTelefone(responsavel.getTelefone());
            }
            if (responsavel.getSenha() != null) {
                responsavelRecuperado.setSenha(passwordEncoder.encode(responsavel.getSenha()));
            }
            return responsavelRepository.save(responsavelRecuperado);
        }
```

```
            return null;
        }
    }
```

services/CriancaService.java

```java
package com.treino.senai.services;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.treino.senai.models.Crianca;
import com.treino.senai.repositories.CriancaRepository;

@Service
public class CriancaService {

    @Autowired
    private CriancaRepository criancaRepository;

    @Autowired
    private ResponsavelService responsavelService;

    public Long contarCriancas() {
        return criancaRepository.count();
    }

    public Crianca buscarCrianca(Integer id) {
        return criancaRepository.findById(id).get();
    }

    public List<Crianca> listarCriancas() {
        return criancaRepository.findAll();
    }

    public Boolean deletarCrianca(Integer id) {
        if (criancaRepository.existsById(id)) {
            criancaRepository.deleteById(id);
            return true;
        }
        return false;
    }

    // a criança é cadastrada já associada a um responsável existente
    public Crianca cadastrarCrianca(Crianca crianca) {
        crianca.setResponsavel(
                responsavelService.buscarResponsavel(crianca.getResponsavel().getId()));
        return criancaRepository.save(crianca);
    }

    public Crianca atualizarCrianca(Integer id, Crianca crianca) {
        Crianca criancaRecuperada = buscarCrianca(id);
        if (criancaRecuperada != null) {
            if (crianca.getNome() != null) {
                criancaRecuperada.setNome(crianca.getNome());
            }
            if (crianca.getDataNascimento() != null) {
                criancaRecuperada.setDataNascimento(crianca.getDataNascimento());
            }
            if (crianca.getResponsavel() != null) {
                criancaRecuperada.setResponsavel(
                        responsavelService.buscarResponsavel(crianca.getResponsavel().getId()));
            }
            return criancaRepository.save(criancaRecuperada);
        }
        return null;
    }
}
```

services/PsicologoService.java

```java
package com.treino.senai.services;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import com.treino.senai.models.Psicologo;
import com.treino.senai.repositories.PsicologoRepository;

@Service
public class PsicologoService {

    @Autowired
    private PsicologoRepository psicologoRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    public Long contarPsicologos() {
        return psicologoRepository.count();
    }

    public Psicologo buscarPsicologo(Integer id) {
        return psicologoRepository.findById(id).get();
    }

    public List<Psicologo> listarPsicologos() {
        return psicologoRepository.findAll();
    }

    public Boolean deletarPsicologo(Integer id) {
        if (psicologoRepository.existsById(id)) {
            psicologoRepository.deleteById(id);
            return true;
        }
        return false;
    }

    public Psicologo cadastrarPsicologo(Psicologo psicologo) {
        psicologo.setSenha(passwordEncoder.encode(psicologo.getSenha()));
        return psicologoRepository.save(psicologo);
    }

    public Psicologo atualizarPsicologo(Integer id, Psicologo psicologo) {
        Psicologo psicologoRecuperado = buscarPsicologo(id);
        if (psicologoRecuperado != null) {
            if (psicologo.getNome() != null) {
                psicologoRecuperado.setNome(psicologo.getNome());
            }
            if (psicologo.getLogin() != null) {
                psicologoRecuperado.setLogin(psicologo.getLogin());
            }
            if (psicologo.getDescricao() != null) {
                psicologoRecuperado.setDescricao(psicologo.getDescricao());
            }
            if (psicologo.getSenha() != null) {
                psicologoRecuperado.setSenha(passwordEncoder.encode(psicologo.getSenha()));
            }
            return psicologoRepository.save(psicologoRecuperado);
        }
        return null;
    }
}
```

services/AgendamentoService.java

```java
package com.treino.senai.services;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.treino.senai.models.Agendamento;
import com.treino.senai.repositories.AgendamentoRepository;

@Service
public class AgendamentoService {

    @Autowired
    private AgendamentoRepository agendamentoRepository;

    @Autowired
    private CriancaService criancaService;

    @Autowired
    private PsicologoService psicologoService;

    public Long contarAgendamentos() {
        return agendamentoRepository.count();
    }

    public Agendamento buscarAgendamento(Integer id) {
        return agendamentoRepository.findById(id).get();
    }

    // ordenados por data e horário
    public List<Agendamento> listarAgendamentos() {
        return agendamentoRepository.findAllByOrderByDataAtendimentoAscHorarioAsc();
    }

    public Boolean deletarAgendamento(Integer id) {
        if (agendamentoRepository.existsById(id)) {
            agendamentoRepository.deleteById(id);
            return true;
        }
        return false;
    }

    public Agendamento cadastrarAgendamento(Agendamento agendamento) {
        agendamento.setCrianca(
                criancaService.buscarCrianca(agendamento.getCrianca().getId()));
        agendamento.setPsicologo(
                psicologoService.buscarPsicologo(agendamento.getPsicologo().getId()));
        return agendamentoRepository.save(agendamento);
    }

    public Agendamento atualizarAgendamento(Integer id, Agendamento agendamento) {
        Agendamento agendamentoRecuperado = buscarAgendamento(id);
        if (agendamentoRecuperado != null) {
            if (agendamento.getDataAtendimento() != null) {
                agendamentoRecuperado.setDataAtendimento(agendamento.getDataAtendimento());
            }
            if (agendamento.getHorario() != null) {
                agendamentoRecuperado.setHorario(agendamento.getHorario());
            }
            if (agendamento.getCrianca() != null) {
                agendamentoRecuperado.setCrianca(
                        criancaService.buscarCrianca(agendamento.getCrianca().getId()));
            }
            if (agendamento.getPsicologo() != null) {
                agendamentoRecuperado.setPsicologo(
```

```
                        psicologoService.buscarPsicologo(agendamento.getPsicologo().getId()));
            }
            return agendamentoRepository.save(agendamentoRecuperado);
        }
        return null;
    }
}
```

Sobre o .get()

No padrão do curso, `findById(id).get()` devolve o objeto. Se o id não existir, o Java lança erro e a API responde 500. Para a prova isso é aceitável; se sobrar tempo, troque por `.orElse(null)` e trate o null no controller.

13. Camada de controle (controllers)

Controllers são a **porta de entrada da API**. Seguem o molde do UsuarioController do curso: caminhos como */salvar-x*, */listar-xs*, */buscar-x/{id}*. Nome: **Entidade + Controller**.

Anotação	Função
@RestController	Define a classe como controlador REST (devolve JSON)
@RequestMapping("/responsavel")	Caminho base de todos os endpoints da classe
@GetMapping / @PostMapping / @PutMapping / @DeleteMapping	Método HTTP + caminho do endpoint
@PathVariable	Valor na URL: /buscar-responsavel/**{id}**
@RequestParam	Valor na query: /listar-responsaveis**?termo=ana**
@RequestBody	Objeto enviado em JSON no corpo da requisição
@Autowired	Traz o service para dentro do controller

controllers/ResponsavelController.java

```java
package com.treino.senai.controllers;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import com.treino.senai.models.Responsavel;
import com.treino.senai.services.ResponsavelService;

@RestController
@RequestMapping("/responsavel")
public class ResponsavelController {

    @Autowired
    private ResponsavelService responsavelService;

    @GetMapping("/contar-responsaveis")
    public Long contarResponsaveis() {
        return responsavelService.contarResponsaveis();
    }

    @GetMapping("/buscar-responsavel/{id}")
    public Responsavel buscarResponsavel(@PathVariable Integer id) {
        return responsavelService.buscarResponsavel(id);
    }

    // /listar-responsaveis  ou  /listar-responsaveis?termo=ana
    @GetMapping("/listar-responsaveis")
    public List<Responsavel> listarResponsaveis(@RequestParam(required = false) String termo) {
        return responsavelService.listarResponsaveis(termo);
    }

    @DeleteMapping("/deletar-responsavel/{id}")
    public String deletarResponsavel(@PathVariable Integer id) {
        if (responsavelService.deletarResponsavel(id)) {
            return "Responsável removido com sucesso.";
        }
        return "Falha ao remover responsável.";
    }

    @PostMapping("/salvar-responsavel")
    public Responsavel cadastrarResponsavel(@RequestBody Responsavel responsavel) {
        return responsavelService.cadastrarResponsavel(responsavel);
    }

    @PutMapping("/atualizar-responsavel/{id}")
    public String atualizarResponsavel(@PathVariable Integer id,
                                       @RequestBody Responsavel responsavel) {
        if (responsavelService.atualizarResponsavel(id, responsavel) != null) {
            return "Responsável atualizado com sucesso.";
        }
        return "Falha ao atualizar responsável.";
    }
}
```

controllers/CriancaController.java

```java
package com.treino.senai.controllers;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.treino.senai.models.Crianca;
import com.treino.senai.services.CriancaService;

@RestController
@RequestMapping("/crianca")
public class CriancaController {

    @Autowired
    private CriancaService criancaService;

    @GetMapping("/contar-criancas")
    public Long contarCriancas() {
        return criancaService.contarCriancas();
    }

    @GetMapping("/buscar-crianca/{id}")
    public Crianca buscarCrianca(@PathVariable Integer id) {
        return criancaService.buscarCrianca(id);
    }

    @GetMapping("/listar-criancas")
    public List<Crianca> listarCriancas() {
        return criancaService.listarCriancas();
    }

    @DeleteMapping("/deletar-crianca/{id}")
    public String deletarCrianca(@PathVariable Integer id) {
        if (criancaService.deletarCrianca(id)) {
            return "Criança removida com sucesso.";
        }
        return "Falha ao remover criança.";
    }

    @PostMapping("/salvar-crianca")
    public Crianca cadastrarCrianca(@RequestBody Crianca crianca) {
        return criancaService.cadastrarCrianca(crianca);
    }

    @PutMapping("/atualizar-crianca/{id}")
    public String atualizarCrianca(@PathVariable Integer id, @RequestBody Crianca crianca) {
        if (criancaService.atualizarCrianca(id, crianca) != null) {
            return "Criança atualizada com sucesso.";
        }
        return "Falha ao atualizar criança.";
    }
}
```

controllers/AgendamentoController.java

```java
package com.treino.senai.controllers;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.treino.senai.models.Agendamento;
import com.treino.senai.services.AgendamentoService;

@RestController
@RequestMapping("/agendamento")
public class AgendamentoController {

    @Autowired
    private AgendamentoService agendamentoService;

    @GetMapping("/contar-agendamentos")
    public Long contarAgendamentos() {
        return agendamentoService.contarAgendamentos();
    }

    @GetMapping("/buscar-agendamento/{id}")
    public Agendamento buscarAgendamento(@PathVariable Integer id) {
        return agendamentoService.buscarAgendamento(id);
    }

    @GetMapping("/listar-agendamentos")
    public List<Agendamento> listarAgendamentos() {
        return agendamentoService.listarAgendamentos();
    }

    @DeleteMapping("/deletar-agendamento/{id}")
    public String deletarAgendamento(@PathVariable Integer id) {
        if (agendamentoService.deletarAgendamento(id)) {
            return "Agendamento removido com sucesso.";
        }
        return "Falha ao remover agendamento.";
    }

    @PostMapping("/salvar-agendamento")
    public Agendamento cadastrarAgendamento(@RequestBody Agendamento agendamento) {
        return agendamentoService.cadastrarAgendamento(agendamento);
    }

    @PutMapping("/atualizar-agendamento/{id}")
    public String atualizarAgendamento(@PathVariable Integer id,
                                       @RequestBody Agendamento agendamento) {
        if (agendamentoService.atualizarAgendamento(id, agendamento) != null) {
            return "Agendamento atualizado com sucesso.";
        }
        return "Falha ao atualizar agendamento.";
    }
}
```

controllers/PsicologoController.java

```java
package com.treino.senai.controllers;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.treino.senai.models.Psicologo;
import com.treino.senai.services.PsicologoService;

@RestController
@RequestMapping("/psicologo")
public class PsicologoController {

    @Autowired
    private PsicologoService psicologoService;

    @GetMapping("/contar-psicologos")
    public Long contarPsicologos() {
        return psicologoService.contarPsicologos();
    }

    @GetMapping("/buscar-psicologo/{id}")
    public Psicologo buscarPsicologo(@PathVariable Integer id) {
        return psicologoService.buscarPsicologo(id);
    }

    @GetMapping("/listar-psicologos")
    public List<Psicologo> listarPsicologos() {
        return psicologoService.listarPsicologos();
    }

    @DeleteMapping("/deletar-psicologo/{id}")
    public String deletarPsicologo(@PathVariable Integer id) {
        if (psicologoService.deletarPsicologo(id)) {
            return "Psicólogo removido com sucesso.";
        }
        return "Falha ao remover psicólogo.";
    }

    @PostMapping("/salvar-psicologo")
    public Psicologo cadastrarPsicologo(@RequestBody Psicologo psicologo) {
        return psicologoService.cadastrarPsicologo(psicologo);
    }

    @PutMapping("/atualizar-psicologo/{id}")
    public String atualizarPsicologo(@PathVariable Integer id, @RequestBody Psicologo psicologo) {
        if (psicologoService.atualizarPsicologo(id, psicologo) != null) {
            return "Psicólogo atualizado com sucesso.";
        }
        return "Falha ao atualizar psicólogo.";
    }
}
```

> **Ao copiar um controller para outra entidade**
>
> Troque **tudo**: classe, caminho, nomes dos métodos, mensagens e variáveis. Mensagens como "Usuário removido" dentro de outro controller foram erro da prova anterior. Use Ctrl + F para procurar o nome antigo.

14. Segurança com JWT (login)

JWT é um token assinado que a API entrega no login. O cliente o envia depois em toda requisição no cabeçalho `Authorization: Bearer <token>`. A API não guarda sessão (stateless).

Fluxo

```
1. POST /auth/login {login, senha}
        -> confere a senha (BCrypt) e gera o token
        <- 200 {token, nome, login, perfil}          (senha errada: 401)

2. GET /responsavel/listar-responsaveis  +  Authorization: Bearer <token>
        -> JwtAuthFilter valida o token e autentica o usuário
        <- 200 com os dados   (sem token ou token inválido: 401)
```

Classe	Pacote	Papel
JwtService	services	Gera o token, extrai o login e valida
CustomUserDetailsService	configs	Ensina o Spring Security a buscar o usuário (tabela psicologo)
JwtAuthFilter	configs	Lê o Bearer token a cada requisição e autentica
SecurityConfig	configs	Rotas públicas, BCrypt, CORS e política stateless
LoginRequestDTO / LoginResponseDTO	dtos	Dados de entrada e de saída do login
AuthController	controllers	Endpoints de login e logout

dtos/LoginRequestDTO.java

```java
package com.treino.senai.dtos;

public class LoginRequestDTO {

    private String login;
    private String senha;

    public LoginRequestDTO() {
    }

    public String getLogin() { return login; }
    public void setLogin(String login) { this.login = login; }
    public String getSenha() { return senha; }
    public void setSenha(String senha) { this.senha = senha; }
}
```

dtos/LoginResponseDTO.java

```java
package com.treino.senai.dtos;

// devolvido no login: token + nome do usuário logado
public class LoginResponseDTO {

    private String token;
    private String nome;
    private String login;
    private String perfil;

    public LoginResponseDTO(String token, String nome, String login, String perfil) {
        this.token = token;
        this.nome = nome;
        this.login = login;
        this.perfil = perfil;
    }

    public String getToken() { return token; }
    public String getNome() { return nome; }
    public String getLogin() { return login; }
    public String getPerfil() { return perfil; }
}
```

services/JwtService.java

```java
package com.treino.senai.services;

import java.nio.charset.StandardCharsets;
import java.util.Date;

import javax.crypto.SecretKey;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;

@Service
public class JwtService {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration}")
    private long expiracao;

    private SecretKey chave() {
        return Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    }

    public String gerarToken(String login, String perfil) {
        return Jwts.builder()
                .subject(login)
                .claim("perfil", perfil)
                .issuedAt(new Date())
                .expiration(new Date(System.currentTimeMillis() + expiracao))
                .signWith(chave())
                .compact();
    }

    public String extrairLogin(String token) {
        return Jwts.parser().verifyWith(chave()).build()
                .parseSignedClaims(token).getPayload().getSubject();
    }

    public boolean isValido(String token) {
        try {
            Jwts.parser().verifyWith(chave()).build().parseSignedClaims(token);
            return true;
        } catch (Exception e) {
            return false;
        }
    }
}
```

> **Os nomes precisam bater**
>
> `@Value("${jwt.secret}")` lê a propriedade `jwt.secret` do application.properties; o mesmo vale para `jwt.expiration`. Se escrever um nome diferente, a aplicação não sobe (*Could not resolve placeholder*). Esse foi um dos erros do PDF de arquitetura.

configs/CustomUserDetailsService.java

```java
package com.treino.senai.configs;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import com.treino.senai.models.Psicologo;
import com.treino.senai.repositories.PsicologoRepository;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    @Autowired
    private PsicologoRepository psicologoRepository;

    @Override
    public UserDetails loadUserByUsername(String login) throws UsernameNotFoundException {
        Psicologo p = psicologoRepository.findByLogin(login)
                .orElseThrow(() -> new UsernameNotFoundException("Usuário não encontrado"));
        return User.builder()
                .username(p.getLogin())
                .password(p.getSenha())
                .roles(p.getDescricao())
                .build();
    }
}
```

configs/JwtAuthFilter.java

```java
package com.treino.senai.configs;

import java.io.IOException;
import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import com.treino.senai.repositories.PsicologoRepository;
import com.treino.senai.services.JwtService;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    @Autowired
    private JwtService jwtService;

    @Autowired
    private PsicologoRepository psicologoRepository;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain chain) throws ServletException, IOException {
        String header = request.getHeader("Authorization");

        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            if (jwtService.isValido(token)) {
                String login = jwtService.extrairLogin(token);
                psicologoRepository.findByLogin(login).ifPresent(p -> {
                    var auth = new UsernamePasswordAuthenticationToken(p.getLogin(), null,
                            List.of(new SimpleGrantedAuthority("ROLE_" + p.getDescricao())));
                    SecurityContextHolder.getContext().setAuthentication(auth);
                });
            }
        }
        chain.doFilter(request, response);
    }
}
```

configs/SecurityConfig.java

```java
package com.treino.senai.configs;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpStatus;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.*;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.HttpStatusEntryPoint;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Autowired
    private JwtAuthFilter jwtAuthFilter;

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config)
            throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.csrf(csrf -> csrf.disable())
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .exceptionHandling(e -> e.authenticationEntryPoint(
                    new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/auth/login", "/swagger-ui/**", "/v3/api-docs/**").permitAll()
                .anyRequest().authenticated())
            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOrigins(
                List.of("http://127.0.0.1:5500", "http://localhost:5500"));
        configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
        configuration.setAllowedHeaders(List.of("*"));

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }
}
```

Trecho	Efeito
permitAll() em /auth/login, /swagger-ui/**, /v3/api-docs/**	Qualquer um acessa (login e documentação)
anyRequest().authenticated()	Todo o resto exige token válido
STATELESS + csrf desabilitado	Sem sessão: autenticação só pelo token
HttpStatusEntryPoint(401)	Sem token ou token inválido responde 401
CORS (portas 5500)	Permite o front-end do Live Server chamar a API

> **Se o enunciado pedir regras por perfil**
>
> Acrescente antes do *anyRequest*, por exemplo: `.requestMatchers("/psicologo/**").hasRole("ADMIN")`. Use `/**` no final: caminho sem isso só casa com a URL exata. E, se precisar criar o primeiro usuário pela API, libere temporariamente `/psicologo/salvar-psicologo` com permitAll.

controllers/AuthController.java

```java
package com.treino.senai.controllers;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.treino.senai.dtos.LoginRequestDTO;
import com.treino.senai.dtos.LoginResponseDTO;
import com.treino.senai.models.Psicologo;
import com.treino.senai.repositories.PsicologoRepository;
import com.treino.senai.services.JwtService;

@RestController
@RequestMapping("/auth")
public class AuthController {

    @Autowired
    private AuthenticationManager authManager;

    @Autowired
    private PsicologoRepository psicologoRepository;

    @Autowired
    private JwtService jwtService;

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequestDTO request) {
        try {
            authManager.authenticate(new UsernamePasswordAuthenticationToken(
                    request.getLogin(), request.getSenha()));
            Psicologo p = psicologoRepository.findByLogin(request.getLogin()).get();
            String token = jwtService.gerarToken(p.getLogin(), p.getDescricao());
            return ResponseEntity.ok(
                    new LoginResponseDTO(token, p.getNome(), p.getLogin(), p.getDescricao()));
        } catch (Exception e) {
            return ResponseEntity.status(401)
                    .body("Falha na autenticação: login ou senha inválidos");
        }
    }

    // JWT não guarda sessão: sair = o cliente descartar o token
    @PostMapping("/logout")
    public String logout() {
        SecurityContextHolder.clearContext();
        return "Logout realizado com sucesso";
    }
}
```

Requisitos de login atendidos

Falha de autenticação: o try/catch devolve 401 com mensagem. **Nome do usuário logado:** o login devolve *nome* no JSON. **Sair do sistema:** endpoint de logout + descartar o token no cliente.

15. Swagger / OpenAPI

O Swagger documenta e permite **testar a API no navegador**, sem ferramenta externa. A classe abaixo define o título e a descrição, e acrescenta o botão **Authorize** para colar o token JWT.

configs/Swagger.java

```java
package com.treino.senai.configs;

import org.springframework.context.annotation.Configuration;

import io.swagger.v3.oas.annotations.OpenAPIDefinition;
import io.swagger.v3.oas.annotations.enums.SecuritySchemeType;
import io.swagger.v3.oas.annotations.info.Info;
import io.swagger.v3.oas.annotations.security.SecurityRequirement;
import io.swagger.v3.oas.annotations.security.SecurityScheme;

@Configuration
@OpenAPIDefinition(
    info = @Info(
        title = "API Clínica",
        version = "1.0",
        description = "API para o sistema de atendimento infantil"),
    security = @SecurityRequirement(name = "bearerAuth"))
@SecurityScheme(name = "bearerAuth", type = SecuritySchemeType.HTTP,
                scheme = "bearer", bearerFormat = "JWT")
public class Swagger {
}
```

Troque *title* e *description* pelo nome e descrição da API do enunciado.

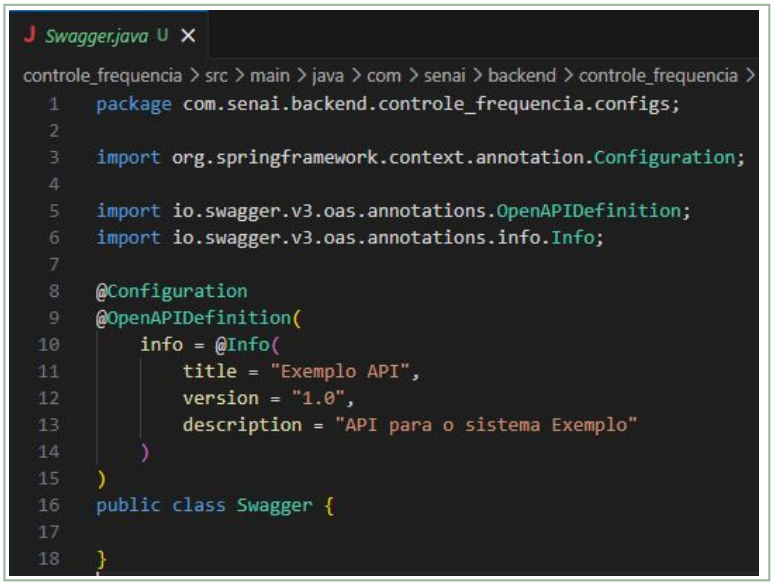


Figura 34 — Swagger.java (exemplo do material de apoio)

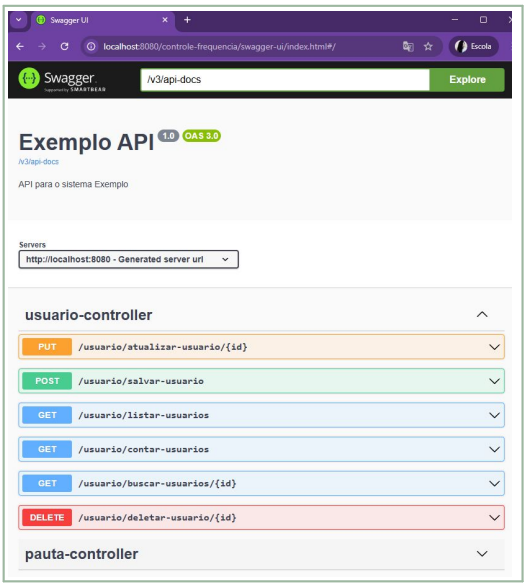


Figura 35 — Swagger UI com os controllers

> **Endereço do Swagger**
>
> `http://localhost:8080/swagger-ui/index.html`. Se o material do curso mandar mudar o caminho com *springdoc.swagger-ui.path*, libere esse caminho no SecurityConfig.

16. Executar e testar a API

Rode `SenaiApplication.java` (Run Java). No terminal deve aparecer *Started SenaiApplication in … seconds* e a sua mensagem de sucesso. Com *show-sql=true* você também vê os comandos SQL a cada requisição.

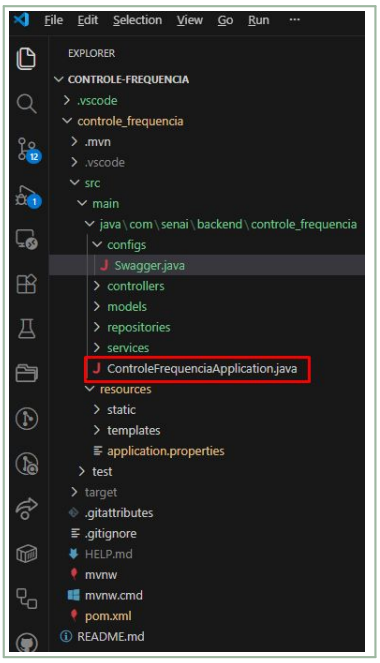


Figura 36 — Árvore do projeto com a aplicação principal

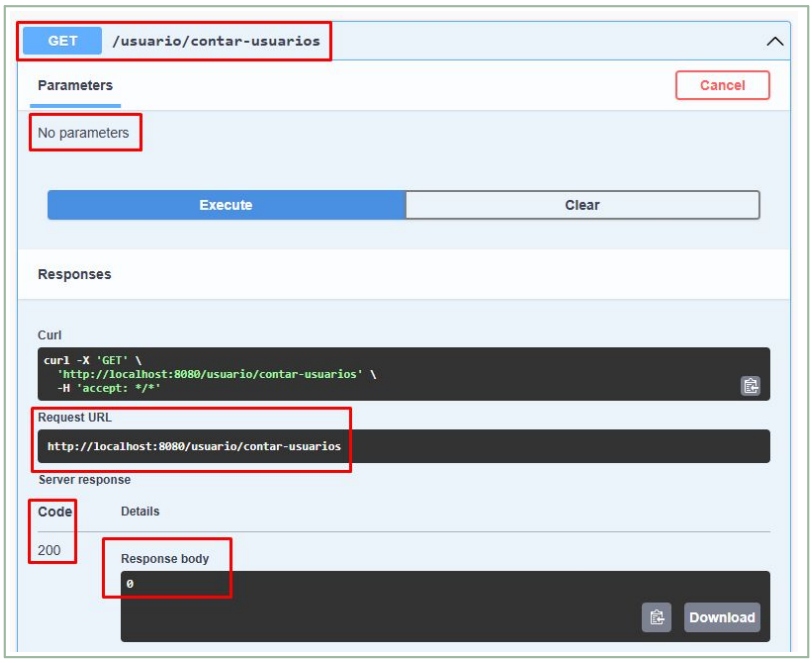


Figura 37 — Testando um endpoint no Swagger (Try it out / Execute)

16.1 Testar pelo Swagger UI

1. Abra `http://localhost:8080/swagger-ui/index.html`.
2. Execute **POST /auth/login** (Try it out) com o JSON de login e copie o valor de **token** da resposta.
3. Clique no botão **Authorize** (cadeado), cole **somente o token** (sem a palavra Bearer) e confirme.
4. Teste cada endpoint: **Try it out → preencher parâmetros/corpo → Execute** e confira o *Code* e o *Response body*. Teste os métodos na ordem: salvar, listar, buscar, atualizar, deletar.

16.2 Testar pelo Postman

- Crie uma coleção e uma variável `token`.
- Em cada requisição: aba **Authorization → Bearer Token → {{token}}**.
- No POST /auth/login, na aba **Tests**, cole o script abaixo para salvar o token automaticamente.
- Confirme que a URL usa a mesma porta do *server.port* (8080).

Postman — aba Tests do login

```
pm.collectionVariables.set("token", pm.response.json().token);
```

16.3 Corpos JSON de exemplo

Corpos de requisição

```
// POST /auth/login
{ "login": "admin", "senha": "admin123" }

// POST /responsavel/salvar-responsavel
{ "nome": "Antônio Soares", "login": "asoares", "senha": "123456", "telefone": "24988887777" }

// PUT /responsavel/atualizar-responsavel/4   (só o que vier no JSON é alterado)
{ "nome": "Antônio Soares Atualizado" }

// POST /crianca/salvar-crianca   (responsavel.id precisa existir)
{ "nome": "Juliana Freitas", "dataNascimento": "2019-05-20", "responsavel": { "id": 2 } }

// PUT /crianca/atualizar-crianca/4
{ "responsavel": { "id": 3 } }

// POST /agendamento/salvar-agendamento   (data: AAAA-MM-DD, horário: HH:mm)
{ "dataAtendimento": "2026-11-10", "horario": "14:30",
  "crianca": { "id": 2 }, "psicologo": { "id": 1 } }

// PUT /agendamento/atualizar-agendamento/4
{ "psicologo": { "id": 2 } }
```

16.4 Roteiro de testes (nesta ordem)

#	Requisição	Resultado esperado
1	POST /auth/login com um usuário válido	200 + token, nome, login e perfil
2	POST /auth/login com senha errada	**401** + mensagem de falha
3	GET /responsavel/listar-responsaveis **sem token**	**401**
4	GET /responsavel/listar-responsaveis com token	200 + lista (sem o campo senha)
5	GET /responsavel/listar-responsaveis?termo=an	200 + só quem tem "an" no nome
6	POST /responsavel/salvar-responsavel	200 + objeto criado, com id
7	PUT /responsavel/atualizar-responsavel/{id}	"atualizado com sucesso"; **conferir que não criou registro novo** (contar-responsaveis)
8	GET /responsavel/buscar-responsavel/{id}	200 + o responsável alterado
9	POST /crianca/salvar-crianca com responsavel.id válido	200 + criança com o responsável
10	POST /agendamento/salvar-agendamento e GET /agendamento/listar-agendamentos	Lista ordenada por data/horário com criança, responsável e psicólogo
11	DELETE /crianca/deletar-crianca/{id}	"removida com sucesso"
12	POST /auth/logout	Mensagem de sucesso; descarte o token

> **Teste de sanidade do PUT**
>
> Chame *contar-responsaveis* antes e depois de um PUT. Se o número aumentou, o seu atualizar está fazendo INSERT (erro do setId). Em qualquer endpoint, olhe também o *console*: com show-sql=true você vê o SQL executado.

17. Commit e push para o GitHub

Configure o Git uma única vez (terminal Git Bash do VS Code):

Configuração inicial

```
git config --global user.name "seunomedeusuariodogithub"
git config --global user.email "seuemaildeusuariodogithub"
```

1. Na lateral esquerda clique em **Source Control**.
2. Escreva uma **mensagem** que diga o que foi feito (ex.: *"API clínica com JWT e CRUDs"*).
3. Passe o mouse em **Changes** e clique em **+** (Stage All Changes). Confira que Changes ficou vazio e **Staged Changes** recebeu os arquivos.
4. Clique na seta ao lado de **Commit** e escolha **Commit & Push**.
5. No navegador, atualize a página do repositório e confirme as alterações.

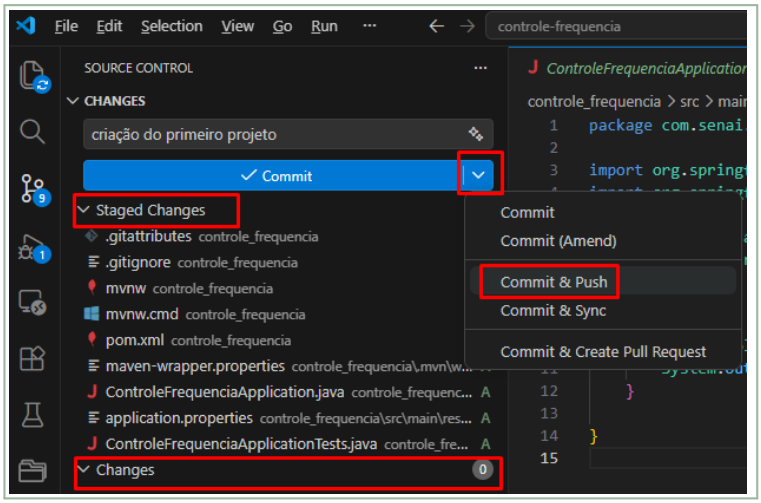


Figura 38 — Staged Changes e Commit & Push

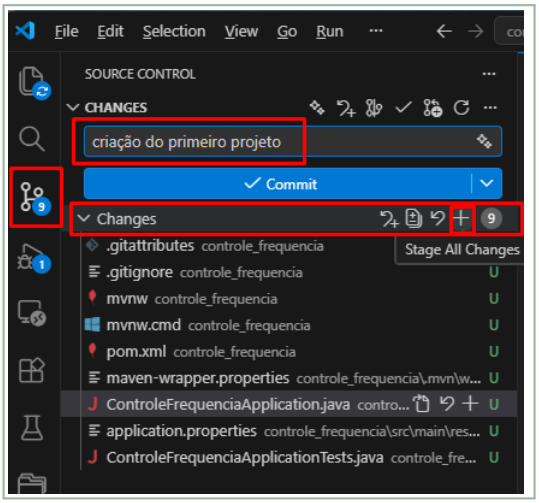


Figura 39 — Source Control com a mensagem de commit

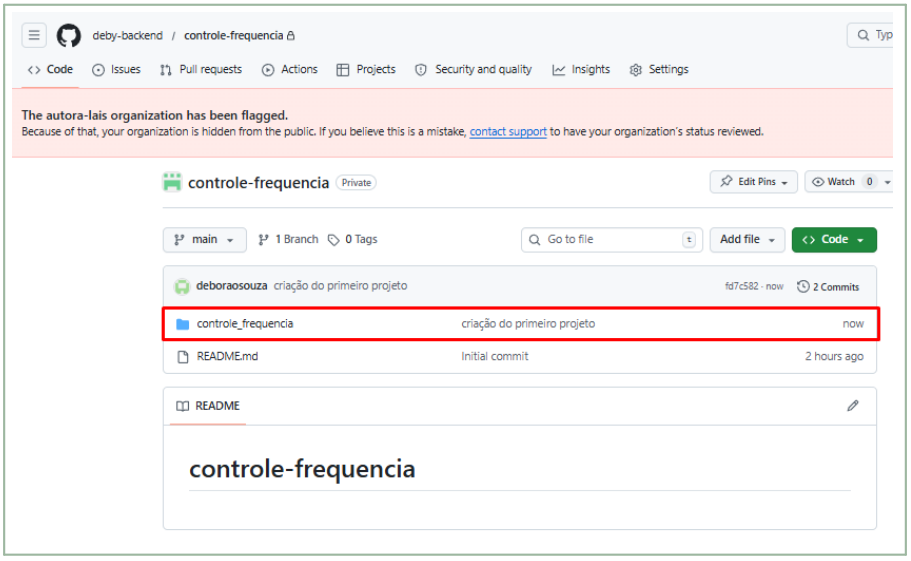


Figura 40 — Repositório atualizado no GitHub

> **Antes de dar push**
>
> Confirme que existe um `.gitignore` ignorando a pasta *target/* (o Initializr já cria). A prova anterior foi entregue com *target/classes* dentro do repositório.

18. Análise dos erros da prova anterior

Comparei, arquivo por arquivo, o projeto entregue com o MER, o script SQL e o enunciado. Os erros estão ordenados por gravidade. Os **críticos** impedem a API de funcionar de verdade (a aplicação até sobe, mas lista vazio, dá 401 ou 500).

22.1 Críticos

#	Onde	Problema	Correção
1	Models x SQL	@Table usa *crianca*, *responsavel*, *agendamento*; o script cria *criancas*, *responsaveis*, *agendamentos*. Com ddl-auto=update o Hibernate cria tabelas novas e vazias e ignora as do script.	@Table idêntico ao script (seção 10)
2	Psicologo	Colunas *psicologo_id*, *username*, *email*, *password*, *role*; o script tem *id*, *nome*, *login*, *senha*, *descricao*. O MER pede **nome**, que a entidade nem tinha.	Mapear id, nome, login, senha, descricao
3	Agendamento	@Column "hora_atendimento"; no script a coluna é *horario*.	@Column(name = "horario")
4	Agendamento	@OneToOne para criança e psicólogo: cria restrição de unicidade e contradiz o MER (1,n). A mesma criança não poderia ter 2 agendamentos.	@ManyToOne
5	Script SQL + login	Senhas inseridas em texto puro (admin123) e o login confere com BCrypt: todo login retorna 401.	Gravar senhas já em BCrypt
6	CriancaService / ResponsavelService	`atualizar` faz `setId(body.getId())`. Sem id no JSON, o save() insere um registro novo; também não atualiza data, responsável, login, telefone.	Buscar pelo id da URL e alterar só campos não nulos
7	Responsavel (requisito)	Listagem sem busca por termo, exigida no enunciado.	findByNomeContainingIgnoreCase + @RequestParam
8	Agendamento (requisito)	Sem ordenação por data; devolve criança e responsável com a senha à mostra.	Método findAllByOrderBy... no repository
9	AuthResponse / login	Devolve só o token: não exibe o nome do usuário logado. Falha de autenticação sem tratamento específico.	LoginResponseDTO com nome + try/catch 401

22.2 Altos e médios

#	Onde	Problema	Correção
10	Crianca	*data_nascimento* como String (coluna é DATE) e atributos com underline.	LocalDate dataNascimento
11	Agendamento	java.sql.Date/Time e atributos *data_atendimento*, getters *getData_atendimento*.	LocalDate / LocalTime, camelCase
12	Responsavel / SQL	telefone INT no script e String no Java (tipos incompatíveis; INT não comporta DDD + 9 dígitos).	VARCHAR(20) + String
13	Services	`findById(id).get()` estoura NoSuchElementException (erro 500) e os testes `!= null` nunca são falsos.	Aceitável no padrão do curso; saiba que id inexistente dá 500
14	PsicologoService / Controller	POST salva senha em texto puro; PUT sobrescreve campos com null; contagem carrega a lista inteira.	BCrypt + atualizar só campos não nulos
15	Entidades expostas	Psicologo implementa UserDetails e é serializado: vazam password, authorities, enabled. Responsavel devolve login e senha.	@JsonProperty WRITE_ONLY na senha
16	AuthController	Endpoint */auth/register* público: qualquer pessoa cria usuário. O enum Role nunca é usado.	Cadastro só por ADMIN
17	SecurityConfig	Rota */produtos/*/imagem* sobrou de outro projeto; nenhuma regra por perfil.	Só as rotas do projeto
18	JwtService	Segredo hex lido como Base64; @Value em campo final e também no construtor.	Segredo em texto, convertido em UTF-8
19	pom.xml	Springdoc 2.2.0 com Spring Boot 4.1.1 (incompatível); dependências duplicadas ou sem uso (starter, jakarta.persistence-api, Thymeleaf, Actuator, Lombok).	Springdoc 3.x; só o necessário
20	Controllers	Mensagens copiadas ("Usuário removido" em criança/agendamento), nomes *contar-responsavels*, *buscResponsavel*; Agendamento sem PUT (import sobrando).	Revisar nomes e completar o CRUD
21	Pastas	*Security* com maiúscula e *enuns* fora do padrão.	configs, enums
22	Script SQL	Datas com hora num campo DATE; sem ponto e vírgula no SELECT final.	Datas AAAA-MM-DD

> **Como nunca mais cair nos erros 1 a 4**
>
> Antes de escrever a primeira entidade, faça uma tabelinha à mão: **tabela do script → classe, coluna → atributo, tipo SQL → tipo Java**. Depois, suba a aplicação com `ddl-auto=validate` (em vez de update): qualquer divergência é apontada no log.

19. Erros encontrados nos materiais de referência

Os materiais de apoio ajudam muito, mas têm pontos que, se copiados sem pensar, quebram o projeto.

Material	Problema	O que fazer
Documentação de arquitetura (JWT)	O *application.properties* define **auth.jwt-secret** e **auth.jwt-expiration-miliseg**, mas o JwtService lê **jwt.secret** e **jwt.expiration-ms**. A aplicação não sobe: *Could not resolve placeholder*.	Nome da propriedade deve ser idêntico nos dois lados
Documentação de arquitetura (SecurityConfig)	Matchers com caminho exato (*/aluno*, */responsavel*) não casam com */aluno/listar*; o controle por perfil FUNCIONARIO nunca é aplicado.	Usar **/recurso/****
Documentação de arquitetura (Postman)	Coleção usa *localhost:8081*, mas *server.port=8080*.	Usar a mesma porta
Documentação de arquitetura (CPF)	O CPF é guardado com BCrypt (hash irreversível): impossível exibir ou buscar. BCrypt é para senha.	Criptografar só senhas
Documentação de arquitetura (AtendimentoRepository)	O SQL nativo com JOIN seleciona colunas extras que não existem na entidade; elas são descartadas, então o JOIN não entrega o que o texto promete.	Método do repository com OrderBy
Slides 12.1	Dizem que o *.get()* "é uma correção do JPA". Na verdade é do Optional e lança exceção se o id não existe.	Entender o comportamento
Slides 16.4 e 16.6	Mandam testar "GET" em atualizar e deletar; os métodos são PUT e DELETE.	Respeitar o verbo HTTP
Slides 13 e 14.1	O passo 13 se chama "Crie os serviços no services", mas trata de controllers. A dependência Springdoc 2.2.0 não casa com o Boot mais atual.	Controllers; Springdoc 3.x no Boot 4

20. Erros comuns e como resolver

Mensagem / sintoma	Causa provável	Solução
Failed to configure a DataSource: url attribute is not specified	application.properties vazio ou fora de src/main/resources	Conferir o arquivo e a pasta
password authentication failed for user "postgres"	Senha do banco diferente da do properties	Ajustar spring.datasource.password
database "x" does not exist	Nome do banco na URL diferente do banco que existe	Corrigir spring.datasource.url
Could not resolve placeholder jwt.secret	Nome da propriedade diferente entre properties e @Value	Padronizar os nomes
WeakKeyException ... 128 bits	jwt.secret com menos de 32 caracteres	Usar segredo longo
relation "x" does not exist / missing table	@Table diferente do script	Corrigir o nome; usar ddl-auto=validate
column "x" does not exist	@Column diferente do script	Corrigir o nome da coluna
Port 8080 was already in use	Outra instância ainda rodando	Parar o terminal anterior ou mudar server.port
404 em todos os endpoints	Controllers fora do pacote base com.treino.senai	Mover as pastas para dentro dele
Login sempre 401	Senha no banco em texto puro, login/senha errados ou usuário inexistente	Cadastrar o usuário pela API (ver abaixo)
401 nas demais rotas	Token ausente, expirado, ou faltou o espaço em "Bearer "	Fazer login de novo e reenviar
403 Forbidden	Perfil sem a role exigida	Entrar como ADMIN ou ajustar a regra
Swagger UI 404 ou erro ao abrir	Springdoc incompatível com o Boot ou caminho bloqueado	Versão 3.x no Boot 4; liberar /swagger-ui/**
JSON parse error (HttpMessageNotReadable)	Data fora do padrão ou vírgula sobrando	Usar "2026-11-10" e "14:30"
violates foreign key constraint (erro 500)	Excluir registro que tem filhos	Excluir primeiro os filhos

Login sempre 401: senha em texto puro

A API compara a senha digitada com um hash **BCrypt** (começa com `$2a$`). Se o usuário existir no banco com senha em texto puro (ex.: admin123), o login nunca passa. A saída sem mexer no código: libere temporariamente `/psicologo/salvar-psicologo` no SecurityConfig (permitAll), cadastre o usuário pelo Swagger (o service já criptografa a senha), faça login com ele e depois remova a liberação.

21. Checklist final antes de entregar

Projeto

- [ ] java --version = 25 e Spring Boot sem sufixo M/RC/SNAPSHOT
- [ ] Pacotes dentro de com.treino.senai, minúsculos e no plural
- [ ] Banco do script existe e a URL do application.properties aponta para ele
- [ ] pom.xml com JJWT e Springdoc compatível com o Boot

Código

- [ ] @Table e @Column idênticos ao script
- [ ] FK com @ManyToOne + @JoinColumn
- [ ] atualizar busca pelo id da URL e não faz setId
- [ ] Busca por termo em responsáveis
- [ ] Agendamentos ordenados por data com criança, responsável e psicólogo
- [ ] Nenhuma senha aparece nas respostas JSON
- [ ] Mensagens e nomes sem sobras de copiar e colar
- [ ] Nenhuma tabela extra criada pelo Hibernate (nomes do @Table iguais ao script)

Segurança

- [ ] Login devolve token e nome; senha errada dá 401
- [ ] Rotas protegidas dão 401 sem token
- [ ] Logout implementado

Testes

- [ ] Todos os endpoints no Swagger ou Postman, com os status esperados
- [ ] Aplicação sobe do zero (parar e rodar de novo)

Entrega

- [ ] .gitignore ignora target/
- [ ] Commit com mensagem clara e push feito
- [ ] Repositório conferido no navegador

> **Dica de prova**
>
> Se faltar tempo, priorize: **(1)** banco e entidades corretos, **(2)** login funcionando, **(3)** CRUDs do enunciado, **(4)** requisitos específicos (busca por termo, ordenação), **(5)** refinamentos.

22. Adaptando este guia para outra prova

A próxima prova será parecida, mas os nomes mudam. Use este roteiro de substituição:

O que muda	Onde mexer
Nome do banco	URL em application.properties e o banco do script
Group Id / Artifact Id / pacote base	Assistente do Initializr; troque *com.treino.senai* em todos os package e import
Entidades e atributos	Pacote models: @Table, @Column e tipos conforme o novo script e MER
Quem faz login	A tabela de usuários do MER: ajuste o repository (findByLogin), CustomUserDetailsService, JwtAuthFilter e AuthController
Perfis (ADMIN, OPERADOR...)	Coluna de perfil do script; se o enunciado exigir, regras hasRole no SecurityConfig
Requisitos especiais (busca, filtro, ordenação)	Repository (nome do método ou @Query), Service e Controller
Nomes dos endpoints	Seguir o enunciado; se não disser, use o padrão do curso: /salvar-x, /listar-xs, /buscar-x/{id}, /atualizar-x/{id}, /deletar-x/{id}
Título do Swagger	OpenApiConfig

Tabela de status HTTP usados

Código	Significado	Quando
200 OK	Sucesso	Todas as operações bem sucedidas, incluindo o login
400 Bad Request	Requisição inválida	JSON malformado ou data no formato errado
401 Unauthorized	Não autenticado	Sem token, token inválido, senha errada
403 Forbidden	Sem permissão	Perfil sem a role exigida (se houver regra hasRole)
404 Not Found	Não encontrado	Caminho de endpoint errado
500 Internal Server Error	Erro no servidor	Id inexistente (.get()), chave estrangeira, erro de mapeamento